



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Thermal-Aware Task Scheduling for Multi-Core Processor Architecture Using Dynamic Temperature Monitoring

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements for
the Course of*

CSA1215 & Computer Architecture

to the award of the degree of

BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE ENGINEERING

(AI&ML), (AI&DS), (IT)

Submitted by

S. Yuva Guru Mani Varma (192425005)

P. Dada Kalander (192424272)

Ganeshwar S (192421342)

Under the Supervision of

Dr. Santha Kumar D

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “Thermal-Aware Task Scheduling for Multi-Core Processor Architecture Using Dynamic Temperature Monitoring” has been carried out by S. Yuva Guru Mani Varma (192425005), P. Dada Kalander (192424272) and Ganeshwar S (192421342) under the supervision of Dr. Santha Kumar D and is submitted in partial fulfilment of the requirements for the current semester of the B. Tech Computer Science Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. V. Sarala,

Professor

CSE

SIMATS Engineering,

SIMATS, Chennai – 602105.

COURSE FACULTY

Dr. Santha Kumar D,

Professor

CSE

SIMATS Engineering,

SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

We, S. Yuva Guru Mani Varma , P. Dada Kalander, Ganeshwar S of the (AI&ML), (AI&DS), (IT), SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “Thermal-Aware Task Scheduling for Multi-Core Processor Architecture Using Dynamic Temperature Monitoring” is the result of our own Bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date:

S. Yuva Guru Mani Varma (192425005)

P. Dada Kalander (192424272)

Ganeshwar S (192421342)

Individual Contribution Statement

Student	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
S. Yuva Guru Mani Varma (192425005)	Module 1: Processor Performance and Thermal Monitoring — Study multi-core processor architecture, monitor CPU utilization, core temperature, frequency, workload and thermal status.	Design and implement processor monitoring module; collect real-time temperature, utilization and frequency data; create monitoring dashboard/data logging mechanism.	Verify sensor/data collection accuracy; analyze processor utilization and temperature variations under different workloads.	Introduction, problem statement, system requirements, Module 1, processor architecture, thermal monitoring methodology.	33%
P. Dada Kalander (192424272)	Module 2: Thermal-Aware Dynamic Task Scheduling — Develop the scheduling algorithm that assigns tasks based on processor temperature, workload and core availability.	Design and implement thermal-aware scheduling algorithm; define temperature thresholds and task allocation logic; implement dynamic task migration/load balancing when a core becomes thermally stressed.	Test scheduling under different workloads and temperature conditions; compare thermal-aware scheduling with conventional scheduling.	Literature survey, proposed methodology, Module 2, scheduling algorithm, flowchart/pseudocode, algorithm implementation.	34%
Ganeshwar S (192421342)	Module 3: Scheduler Integration and Performance Evaluation — Integrate monitoring and scheduling modules; develop the overall system interface and evaluate performance.	Integrate Modules 1 and 2; develop scheduler execution framework/interface; implement performance metrics such as execution time, temperature reduction, CPU utilization, throughput and task completion rate.	Conduct complete system testing; generate graphs and performance comparisons; evaluate thermal efficiency, scheduling overhead and overall system performance.	Module 3, integration architecture, experimental setup, results, performance analysis, conclusion and future enhancement.	33%
Total					100%

ABSTRACT

Single-core processors execute all tasks using a single processing unit, which can lead to increased processor utilization and temperature during continuous or computationally intensive task execution. Traditional task scheduling techniques mainly focus on task execution and processor utilization without considering the real-time thermal condition of the processor. Excessive temperature can result in performance degradation, system instability, thermal throttling, and reduced processor hardware lifespan. Therefore, an engineering approach that integrates thermal conditions into task scheduling is required for safer and more reliable processor operation.

This project proposes a Thermal-Aware Task Scheduling System for Single-Core Processor Architecture Using Dynamic Temperature Monitoring. The proposed system continuously monitors processor temperature during task execution and uses predefined thermal thresholds to make dynamic scheduling decisions. When the processor operates within a safe temperature range, tasks are executed normally. When the temperature approaches a critical level, the scheduler prioritizes lightweight and low-power tasks and can introduce cooling intervals or task delays to reduce thermal stress.

The system is designed and evaluated using a Python-based scheduler simulation environment. Performance parameters including CPU temperature, execution time, CPU utilization, and task completion are monitored and analyzed. The proposed thermal-aware scheduling approach is compared with conventional scheduling to evaluate its effectiveness under different workload and thermal conditions. The implementation uses Python along with libraries such as NumPy, Pandas, Matplotlib, and psutil for scheduling, monitoring, data processing, and visualization.

The results indicate that thermal-aware scheduling can reduce CPU temperature compared with conventional scheduling, minimize overheating conditions, and control performance degradation caused by excessive temperature. The approach also supports safer task execution and improves overall system efficiency and reliability. Although thermal-aware scheduling introduces additional scheduler complexity and may increase task execution time under high-temperature conditions, it provides an effective mechanism for balancing processor performance and thermal safety.

Keywords: Thermal-Aware Scheduling, Dynamic Temperature Monitoring, Single-Core Processor, Task Scheduling, CPU Thermal Management, Processor Reliability

TABLE OF CONTENTS

Sl. No	Title	Page No
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
lii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction	1
2	CHAPTER 2: Literature Review	13
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	18
4	CHAPTER 4: Implementation, Testing & Results, Project Management	27
5	CHAPTER 5: Conclusion & Reflection	39
	References	42
	Appendices	43

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1.	Verification result	31
2.	Implementation img 1	37
3.	Implementation img 2	38
4	Implementation img 3	38

LIST OF TABLES

Table No.	Table Name	Page No.
1.	Verification resulting	16
2.	Stakeholder / user req	18
3.	Functional and non Functional	19
4.	Trade -off Analysis	24

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Single-core processors execute all assigned tasks using a single processing unit, which can result in increased processor utilization and heat generation when multiple or computationally intensive tasks are executed continuously. Traditional task scheduling techniques primarily focus on factors such as execution time, task priority, and processor utilization, while generally not considering the real-time thermal condition of the processor.

As processor temperature increases, the system may experience performance degradation, thermal throttling, instability, and potential reduction in hardware lifespan. Therefore, incorporating temperature information into task scheduling can provide a more reliable approach to processor management.

The proposed project focuses on developing a thermal-aware task scheduling system for a single-core processor using dynamic temperature monitoring. The system continuously monitors processor temperature during task execution and uses predefined thermal thresholds to dynamically influence scheduling decisions. When the processor temperature approaches or exceeds a safe operating limit, lightweight or low-power tasks can be prioritized and cooling intervals or task delays can be introduced. This approach aims to maintain safe processor operation while improving system reliability and performance.

1.2 Need for the Project

Traditional task scheduling approaches do not sufficiently consider the changing thermal condition of a processor during task execution. Continuous execution of computationally intensive workloads can increase CPU temperature, which may negatively affect performance and system stability.

The need for a thermal-aware scheduling system arises from the following factors:

- Temperature control: Continuous monitoring helps identify increasing processor temperature before it reaches a critical level.
- Overheating prevention: Dynamic scheduling can reduce the workload pressure when the processor approaches its thermal limit.
- Performance improvement: Controlling excessive temperature can help reduce performance degradation caused by thermal conditions.

- Hardware reliability: Preventing repeated overheating can contribute to increasing the operational lifespan of processor hardware.
- Dynamic decision-making: Scheduling decisions can be adjusted according to the current thermal state rather than using a fixed scheduling strategy.
- Safe task execution: Thermal conditions can be incorporated into scheduling to maintain stable and efficient processor operation.

Thus, integrating dynamic temperature monitoring with task scheduling provides an engineering approach for balancing task execution, processor performance, temperature, and reliability. These requirements directly support the project's objective of improving performance and reliability under thermal constraints.

1.3 Problem Statement

Traditional task scheduling techniques for single-core processors primarily focus on task execution and processor utilization without considering the processor's real-time temperature. During continuous execution of computationally intensive workloads, the processor temperature can increase significantly, leading to performance degradation, system instability, and potential reduction in hardware lifespan.

The absence of a dynamic mechanism that monitors processor temperature and adjusts task execution accordingly creates a limitation in maintaining both thermal safety and system performance. Therefore, the engineering problem addressed in this project is to develop a thermal-aware task scheduling approach that continuously monitors processor temperature and dynamically modifies task scheduling decisions based on predefined thermal conditions.

The proposed approach aims to prioritize lightweight or low-power tasks and introduce suitable cooling intervals or task delays when the processor temperature approaches unsafe levels, while allowing normal task execution under safe thermal conditions. This enables a balance between processor temperature, execution time, CPU utilization, performance, and reliability.

1.4 Project Aim

The main aim of this project is to develop a thermal-aware task scheduling system for a single-core processor that continuously monitors processor temperature and dynamically adjusts task execution based on real-time thermal conditions. The system aims to prevent processor

overheating while maintaining efficient task execution, improving system reliability, and supporting longer processor hardware lifespan.

1.5 Project Objectives

The objectives of the proposed thermal-aware task scheduling system are:

1. To continuously monitor processor temperature during task execution using dynamic temperature monitoring.
2. To prevent overheating in a single-core processor by considering real-time thermal conditions during scheduling.
3. To dynamically adjust task scheduling decisions based on predefined thermal thresholds.
4. To prioritize lightweight and low-power tasks when the processor temperature becomes high.
5. To introduce suitable cooling intervals or task delays when the processor approaches a critical temperature.
6. To improve processor performance and reliability under thermal constraints.
7. To increase the potential lifespan of processor hardware by reducing excessive thermal stress.
8. To evaluate the proposed scheduling approach by comparing CPU temperature, execution time, CPU utilization, and overall system performance with traditional scheduling methods.

1.6 Scope

The scope of this project is to design and evaluate a thermal-aware task scheduling approach for a single-core processor using dynamic temperature monitoring. The system focuses on monitoring processor temperature during task execution and using the monitored thermal condition to influence scheduling decisions.

The project includes the development of a thermal monitoring mechanism, a thermal-aware dynamic task scheduling algorithm, and the integration and evaluation of the scheduling approach. When the processor temperature remains within the safe range, tasks are executed normally. When the temperature becomes high, the scheduler can prioritize lightweight and low-power tasks and introduce cooling intervals or task delays.

The system is evaluated by monitoring parameters such as CPU temperature, execution time, CPU utilization, and task completion, followed by comparison with traditional scheduling. The implementation is based on a Python-based scheduler simulation environment.

The project is limited to a single-core processor architecture and focuses on temperature-based scheduling. Multi-core processor scheduling, temperature prediction, advanced power optimization, and deployment on mobile or IoT devices are considered outside the present implementation and are identified as areas for future development.

1.7 Expected Engineering Outcome

The expected outcome of this project is a functional thermal-aware task scheduling system for a single-core processor that integrates dynamic temperature monitoring with task scheduling decisions. The system is expected to monitor processor temperature during task execution and dynamically respond to changing thermal conditions.

The developed system is expected to:

- Maintain processor temperature within a safer operating range by considering thermal conditions during scheduling.
- Dynamically prioritize lightweight and low-power tasks when processor temperature becomes high.
- Introduce cooling intervals or task delays when required to reduce thermal stress.
- Measure and evaluate CPU temperature, execution time, CPU utilization, and task completion.
- Provide a comparative evaluation between thermal-aware scheduling and conventional scheduling.
- Improve overall processor reliability and support safer and more efficient task execution.
- Demonstrate the potential of thermal-aware scheduling to reduce overheating and extend processor hardware lifespan.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The literature review for this project was conducted by examining existing research and technical approaches related to thermal-aware task scheduling, processor thermal management, temperature-based scheduling, and dynamic voltage and frequency scaling (DVFS). Relevant research works focusing on single-core processors, embedded processors, mobile devices, and thermal-aware scheduling techniques were considered to understand the existing methods used for controlling processor temperature during task execution.

The reviewed studies were analyzed based on their scheduling approach, thermal monitoring mechanism, workload management strategy, and performance evaluation parameters. Particular attention was given to approaches that use processor temperature as a factor in scheduling decisions. The findings from the review were used to identify limitations in conventional scheduling and to establish the engineering basis for the proposed thermal-aware task scheduling system.

2.2 Review of Existing Work

Existing research on thermal-aware task scheduling focuses on managing processor temperature while maintaining efficient task execution. Several approaches consider thermal conditions as an additional factor in scheduling decisions rather than relying only on conventional scheduling parameters.

Podobas and Brorsson investigated an architecture-aware task scheduling approach that considers thermal conditions during task allocation. Their work demonstrates the importance of considering processor architecture and thermal behavior when scheduling computational workloads.

Kumar and Verma studied heat-aware task scheduling using Dynamic Voltage and Frequency Scaling (DVFS) for embedded processors. Their approach focuses on controlling processor power and heat generation by adjusting operating conditions according to thermal requirements. Murali, Raghunathan, and Jha investigated thermal-aware scheduling for deep-learning workloads on mobile devices. Their work highlights the importance of thermal management for computationally intensive applications where continuous processing can lead to increased device temperature.

Rodríguez, Meumeu Yomsi, and Lipari studied thermal-aware scheduling for DVFS-enabled single-core processors. Their work is particularly relevant to the proposed project because it

addresses the relationship between processor temperature, scheduling decisions, and processor operating conditions.

Zhou and Li investigated temperature-constrained task scheduling for embedded processors, demonstrating the importance of incorporating temperature constraints into task scheduling to improve processor reliability and thermal performance.

These existing studies provide the technical foundation for integrating thermal conditions with task scheduling. However, the proposed project focuses specifically on a single-core processor with dynamic temperature monitoring and temperature-threshold-based task scheduling, where the current thermal condition is directly used to modify task execution and prioritization.

2.4 Research / Engineering Gap

The reviewed existing approaches demonstrate that processor temperature can be incorporated into task scheduling and thermal management. However, conventional task scheduling generally does not use real-time processor temperature as a direct factor in task execution decisions. Existing thermal-aware approaches may also depend on specific techniques such as DVFS or application-specific environments.

A need therefore exists for a simpler dynamic scheduling approach that continuously monitors the processor's current thermal condition and uses predefined temperature thresholds to modify task execution. In particular, there is a need to integrate real-time temperature monitoring, dynamic task prioritization, lightweight task selection, and cooling intervals within a single-core processor scheduling framework.

The engineering gap addressed by this project is the development and evaluation of such a thermal-aware scheduling mechanism, with performance assessed using CPU temperature, execution time, CPU utilization, and task completion. The proposed approach aims to balance thermal safety with efficient processor utilization rather than considering task execution performance alone.

2.3 Comparative Analysis

The existing approaches were compared based on their thermal management capability, scheduling strategy, and relevance to the proposed system.

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Architecture-Aware Task Scheduling	Considers processor architecture during task scheduling and can improve workload management.	Thermal conditions are not the only focus of the scheduling process.	Provides a basis for considering processor characteristics during scheduling.
Heat-Aware Scheduling using DVFS	Helps control processor power and heat by adjusting operating conditions.	Requires DVFS-enabled processor support and additional control mechanisms.	Provides a reference for managing processor heat through scheduling decisions.
Thermal-Aware Scheduling for Mobile Devices	Addresses thermal management for computationally intensive workloads.	Primarily designed for mobile-device environments.	Demonstrates the importance of thermal-aware scheduling for continuous workloads.
Thermal-Aware Scheduling for Single-Core Processors	Directly considers temperature and scheduling for single-core processors.	May depend on DVFS and specific processor operating conditions.	Closely related to the proposed single-core thermal-aware scheduling approach.
Temperature-Constrained Task	Uses temperature constraints to manage task	Focuses mainly on temperature constraints rather	Supports the use of thermal limits in dynamic

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Scheduling	execution and processor reliability.	than the complete proposed monitoring and prioritization approach.	scheduling decisions.
Proposed Thermal-Aware Scheduling	Continuously monitors temperature, dynamically prioritizes tasks, and introduces cooling intervals when required.	Increased scheduling complexity and possible task delays at high temperatures.	Provides the intended integrated approach for single-core thermal-aware task scheduling.

Table :1 Comparative Analysis

The comparison shows that existing studies provide different methods for incorporating thermal conditions into processor management. The proposed system combines dynamic temperature monitoring, thermal threshold-based decision making, dynamic task prioritization, and cooling interval insertion within a single-core scheduling framework.

2.5 Proposed Contribution

The proposed project contributes a thermal-aware task scheduling approach designed specifically for a single-core processor. Unlike conventional scheduling, the proposed system incorporates the processor's current temperature into scheduling decisions through continuous dynamic temperature monitoring.

The main contribution is the integration of thermal monitoring and dynamic task scheduling into a single scheduling framework. The system uses predefined thermal thresholds to determine the appropriate scheduling action. Under safe thermal conditions, tasks are executed normally. When the processor temperature becomes high, lightweight and low-power tasks are

prioritized, while cooling intervals or task delays can be introduced to reduce thermal stress.

The proposed system also provides a comparative evaluation against conventional scheduling by measuring CPU temperature, execution time, CPU utilization, and task completion. This enables the effectiveness of thermal-aware to be evaluated in terms of processor safety, performance, and reliability.

Overall, the contribution of the project is a scheduling mechanism that aims to maintain a balance between thermal safety and efficient task in a single-core processor environment.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

3.1.1 Stakeholder / User Requirements

The proposed system is intended to provide a safe and efficient task scheduling mechanism for a single-core processor by considering its real-time thermal condition.

Stakeholder	Requirement
System User	The system should execute tasks efficiently while maintaining safe processor temperature.
System Administrator	The system should provide reliable monitoring of processor temperature and task execution.
Developer	The system should support dynamic scheduling decisions based on predefined thermal thresholds.
Project Evaluator	The system should provide measurable results for CPU temperature, execution time, CPU utilization, and task completion.

Table 2: Stakeholder / User Requirements

3.1.2 Functional and Non-Functional Requirements

Requirement	Type	Measurable Target
Monitor processor temperature during task execution	Functional	Temperature monitored continuously during execution
Compare temperature with predefined thermal threshold	Functional	Thermal condition checked for every scheduling decision
Dynamically prioritize lightweight tasks	Functional	Lightweight tasks prioritized under high-temperature conditions
Introduce cooling intervals or task delays	Functional	Cooling action applied when temperature approaches danger level
Execute tasks normally under safe conditions	Functional	Normal scheduling maintained within safe temperature range

Requirement	Type	Measurable Target
Measure CPU utilization and execution time	Functional	Parameters recorded during system evaluation
Provide reliable task execution	Non-Functional	Tasks should complete without thermal instability
Maintain system performance under thermal constraints	Non-Functional	Performance degradation should be controlled
Provide clear performance evaluation	Non-Functional	Results represented using measurable data and graphs
Support maintainable implementation	Non-Functional	Modular Python-based implementation

Table 3 : Functional and Non-Functional Requirements

3.2 Constraints & Applicable Standards

The proposed system is developed within the constraints of a single-core processor environment and a Python-based simulation setup. The main constraints are related to temperature monitoring accuracy, scheduling complexity, processor performance, and the limitations of the simulation environment.

Constraint / Standard	Requirement	How Applied in This Project
Single-Core Architecture	Scheduling is designed for a single processing core.	The proposed scheduling algorithm is evaluated using a single-core processor model.
Temperature Monitoring	Processor temperature must be monitored during task execution.	Dynamic temperature monitoring is used to provide the current thermal condition to the scheduler.
Thermal	A predefined safe thermal limit is	The current temperature is compared with

Constraint / Standard	Requirement	How Applied in This Project
Threshold	required for scheduling decisions.	the defined thermal threshold.
Processing Performance	Thermal control should not unnecessarily reduce task execution efficiency.	Lightweight tasks and cooling intervals are introduced only when thermal conditions require them.
Monitoring Accuracy	Temperature measurements should be sufficiently reliable for scheduling decisions.	Temperature data is continuously collected and used for thermal-state evaluation.
Software Environment	The system requires a suitable programming and simulation environment.	Python 3.10 or above with NumPy, Pandas, Matplotlib and psutil is used for implementation and analysis.

No specific external hardware safety standard is identified in the provided project material. Therefore, standards that are not directly applicable to the implemented Python-based scheduling simulation are not claimed as being followed. The project instead focuses on the stated technical constraints and thermal-management requirements.

3.2 Design Specifications

The design specifications define the main technical parameters required for implementing the thermal-aware task scheduling system. These specifications are based on the project requirements and the proposed scheduling approach.

Parameter	Target / Requirement	Unit	Priority
Processor Architecture	Single-core processor	—	High
Temperature Monitoring	Continuous monitoring during task execution	Temperature reading	High
Thermal Threshold	Predefined safe operating limit	°C	High

Parameter	Target / Requirement	Unit	Priority
Task Scheduling	Dynamic scheduling based on thermal condition	—	High
Task Prioritization	Lightweight and low-power tasks during high temperature	—	High
Cooling Control	Introduce cooling intervals or task delays when required	Time interval	Medium
CPU Utilization	Monitor processor utilization during execution	%	High
Task Execution Time	Measure execution time for performance evaluation	seconds	High
Task Completion	Monitor successful task completion	Completed tasks	High
Performance Evaluation	Compare thermal-aware scheduling with conventional scheduling	—	High
Data Visualization	Present temperature and performance results using graphs	—	Medium

The design prioritizes continuous thermal monitoring and dynamic scheduling because these functions directly determine how the system responds to changing processor temperature. CPU temperature, execution time, CPU utilization, and task completion are used as the primary evaluation parameters.

3.3 Alternative Solutions & Evaluation

Alternative 1: Conventional Task Scheduling

Conventional scheduling executes tasks based mainly on scheduling parameters such as task order, priority, or execution requirements without continuously considering processor temperature. It provides a simple scheduling mechanism but does not dynamically respond to thermal conditions.

Alternative 2: DVFS-Based Thermal Management

DVFS-based thermal management controls processor operating conditions by dynamically adjusting voltage and frequency to manage power consumption and heat generation. This approach can provide effective thermal control but requires processor support for dynamic voltage and frequency adjustment.

Alternative 3: Proposed Thermal-Aware Dynamic Task Scheduling

The proposed approach continuously monitors processor temperature and uses thermal thresholds to influence scheduling decisions. When temperature increases, lightweight or low-power tasks are prioritized and cooling intervals or task delays can be introduced to reduce thermal stress.

Criterion	Weight	Alternative 1: Conventional	Alternative 2: DVFS-Based	Alternative 3: Proposed
Performance	25%	High	High	High
Thermal Control	25%	Low	High	High
Implementation Cost	15%	Low	Medium	Low
Safety	15%	Low	High	High
Sustainability	10%	Medium	High	High
Reliability	10%	Medium	High	High

Based on the evaluation, the proposed thermal-aware scheduling approach is selected because it provides direct integration of dynamic temperature monitoring with task scheduling while maintaining a relatively simple Python-based implementation. It also supports dynamic task prioritization and cooling intervals according to the processor's thermal condition.

3.4 Selected Design & Detailed Design

Based on the evaluation of the alternative solutions, the **Thermal-Aware Dynamic Task Scheduling** approach is selected as the final design. The approach is suitable for the project because it directly incorporates processor temperature into task scheduling decisions without requiring a complex hardware-level power management mechanism.

The system consists of three major functional modules. The first module continuously monitors processor temperature, CPU utilization, and task execution time. The measured temperature is compared with a predefined thermal threshold and the current thermal status is provided to the scheduler. The second module uses this thermal information to dynamically prioritize tasks. Lightweight and low-power tasks are given priority when the processor temperature becomes high, while cooling intervals or task delays may be introduced when required. The third module integrates the scheduling mechanism and evaluates the system by measuring temperature, execution time, CPU utilization, and task completion.

System Flow:

Task Input → Temperature Monitoring → Thermal Threshold Check → Scheduling Decision → Task Prioritization / Cooling Interval → Task Execution → Performance Monitoring → Result Analysis

The selected design follows a closed-loop approach in which the processor's current thermal condition influences subsequent scheduling decisions. This allows the system to maintain normal task execution under safe conditions while responding dynamically when the temperature becomes high.

The implementation is developed using a Python-based scheduler simulation environment. Python, NumPy, Pandas, Matplotlib, and psutil are used for scheduling, data processing, visualization, and performance monitoring.

3.6 Trade-off Analysis

The design of the proposed thermal-aware scheduling system involves balancing processor performance, thermal safety, scheduling complexity, and task execution time. The following trade-offs were considered while selecting the final design.

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Temperature-based scheduling	Conventional scheduling	Provides thermal control and responds to processor temperature.	Adds additional scheduling logic.	Selected because thermal condition is a primary requirement of the project.
Dynamic task prioritization	Fixed task priority	Allows lightweight tasks to be selected during high-temperature conditions.	May change the original task execution order.	Selected to reduce thermal stress during high-temperature operation.
Cooling intervals	Continuous task execution	Helps reduce processor temperature and overheating risk.	Can increase task completion time.	Selected when the processor approaches the thermal limit.
Lightweight task preference	Execute all tasks normally	Reduces processor workload during high-temperature conditions.	Computationally intensive tasks may be delayed.	Selected to maintain safer processor operation.
Continuous temperature monitoring	Periodic or no monitoring	Provides current thermal information for scheduling decisions.	Increases monitoring overhead.	Selected because dynamic thermal information is required by the proposed system.
Python-based simulation	Hardware-level implementation	Lower implementation complexity and supports rapid testing and visualization.	Does not represent every hardware-level thermal characteristic.	Selected for development and evaluation within the project scope.

Table 4 : Trad-off analysis

The major trade-off is between **thermal safety and task execution performance**. Introducing cooling intervals and prioritizing lightweight tasks can increase execution time, but these measures help control excessive processor temperature. The final design therefore prioritizes safe and reliable

processor operation while maintaining acceptable scheduling performance. The project also recognizes that high-temperature conditions may delay task execution and that thermal-aware scheduling introduces additional scheduler complexity.

3.7 Sustainability, Ethics, Safety, Risk & Security

The proposed thermal-aware task scheduling system considers sustainability, ethical use, processor safety, operational risks, and security as part of the system design. These considerations are directly related to the objective of reducing excessive processor temperature while maintaining reliable task execution.

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Reducing excessive processor heating can support efficient processor operation and potentially extend hardware lifespan.	CPU temperature and thermal behaviour during task execution.	Thermal-aware scheduling was selected to reduce unnecessary thermal stress.
Ethics	The system should provide reliable and accurate monitoring without manipulating or fabricating performance results.	Recorded temperature and performance measurements.	Results are evaluated using measurable system parameters.
Health & Safety	Excessive processor temperature can lead to instability and hardware damage.	Thermal threshold and temperature monitoring.	Cooling intervals and task delays are introduced when required.
Security	Scheduling and monitoring data should be protected from unauthorized modification during system evaluation.	Scheduler inputs, monitoring data, and evaluation results.	Controlled data handling is considered during implementation and testing.

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Inaccurate temperature measurement	Medium	High	High	Continuously monitor temperature and evaluate readings during execution.	Medium
Processor overheating	Medium	High	High	Use thermal thresholds, lightweight task prioritization, and cooling intervals.	Low
Increased task execution time	High	Medium	High	Apply thermal controls only when required by processor temperature.	Medium
Increased scheduler complexity	Medium	Medium	Medium	Use a modular scheduling approach with clearly defined thermal decisions.	Low
Reduced performance under strict thermal limits	Medium	Medium	Medium	Balance thermal safety with task execution and monitor performance metrics.	Medium

The major safety consideration is processor overheating. The proposed system addresses this risk by continuously monitoring temperature and dynamically modifying task scheduling when the processor approaches unsafe thermal conditions. However, the effectiveness of the approach depends on the accuracy of temperature monitoring and workload characteristics.

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The proposed system is developed using a Python-based scheduler simulation approach. The development process consists of implementing processor performance and thermal monitoring, developing the thermal-aware dynamic task scheduling algorithm, integrating the scheduling mechanism, and evaluating its performance against conventional scheduling.

During execution, processor temperature and performance parameters are monitored and provided to the scheduler. Based on predefined thermal thresholds, the scheduler dynamically determines whether tasks should be executed normally, prioritized according to their workload, or delayed to introduce a cooling interval. The resulting performance data is then collected and analyzed to evaluate the effectiveness of the proposed approach.

Resource	Purpose
Python 3.10 or above	System development and scheduler implementation
NumPy	Numerical processing and data handling
Pandas	Performance data organization and analysis
Matplotlib	Graph generation and result visualization
psutil	Processor performance and monitoring information
Python Standard Library	Core scheduling and program functionality
Windows 10/11 or Ubuntu Linux	Development and execution environment
Visual Studio Code / PyCharm	Code development and testing
Python-based Scheduler Simulation	System implementation and evaluation environment

The selected tools support the development, monitoring, analysis, and visualization requirements of the proposed thermal-aware scheduling system.

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

4. Used

Software Implementation

The proposed thermal-aware task scheduling system is implemented using Python in a simulation-based environment. The scheduler receives task information and continuously evaluates the processor's thermal condition during task execution. The current CPU temperature is compared with a predefined thermal threshold, and the scheduler dynamically selects the appropriate scheduling action.

When the processor temperature remains within the safe range, tasks are executed normally. When the temperature becomes high, the scheduler prioritizes lightweight and low-power tasks. If the temperature approaches the danger level, a cooling interval or task delay can be introduced before continuing execution.

Performance Monitoring

The system monitors CPU temperature, CPU utilization, and task execution time during the scheduling process. The collected data is used to evaluate the thermal behaviour and performance of the proposed approach.

Data Analysis and Visualization

Pandas and NumPy are used for organizing and processing the collected performance data, while Matplotlib is used to generate graphs for analysing CPU temperature and scheduling performance. The proposed approach is evaluated by comparing its results with conventional scheduling.

Tool / Software / Equipment	Purpose	Stage Used
Python	Scheduler development and implementation	Development
psutil	CPU performance and temperature monitoring	Monitoring
NumPy	Numerical data processing	Analysis
Pandas	Data organization and analysis	Analysis
Matplotlib	Graph generation and visualization	Results

Tool / Software / Equipment	Purpose	Stage Used
Visual Studio Code / PyCharm	Program development and testing	Development
Python-based Scheduler Simulation	Task scheduling and system evaluation	Testing

The project material specifies these software tools for implementing the scheduler, monitoring processor performance, processing data, and presenting evaluation results.

4.3 Test Plan & Verification Results

The testing process is designed to verify whether the proposed thermal-aware task scheduling system meets its intended thermal management and performance requirements. The system is tested by monitoring processor temperature during task execution and evaluating how the scheduler responds to different thermal conditions.

Test Plan

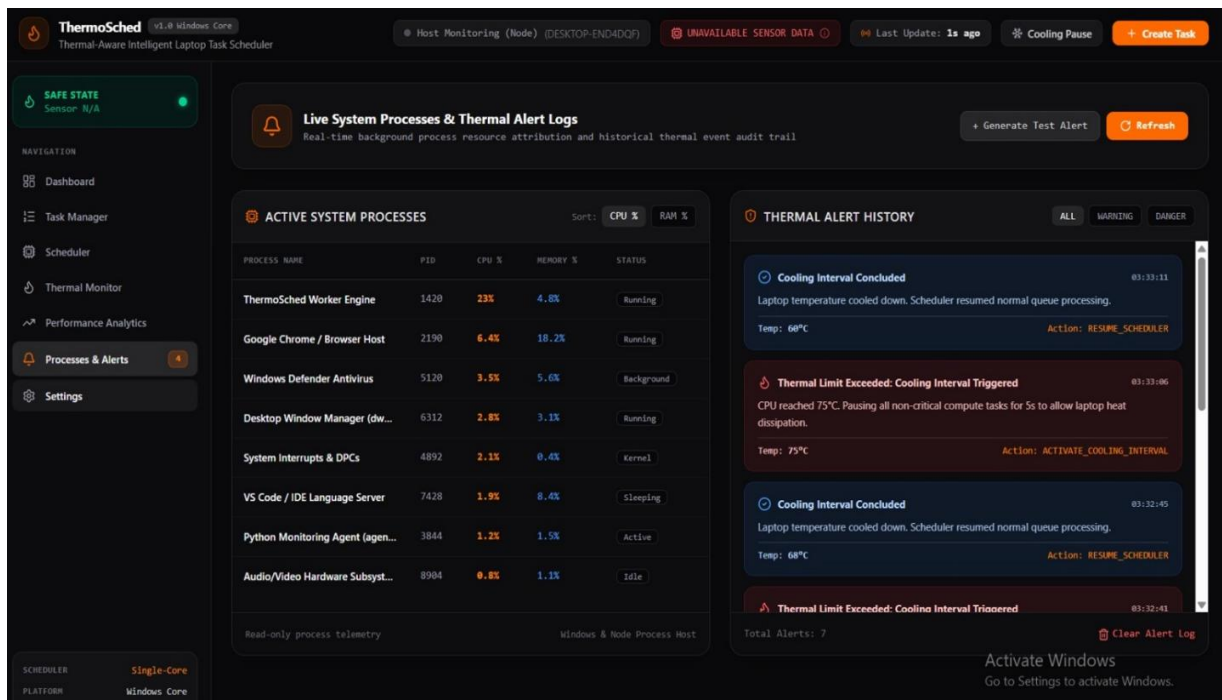
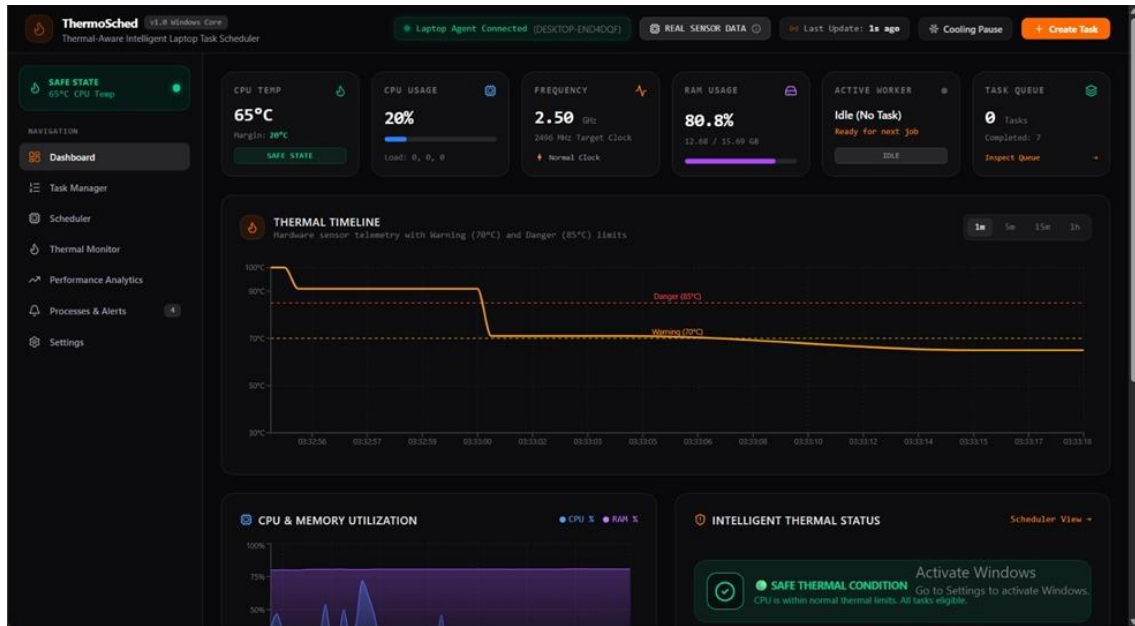
Requirement	Test Method	Acceptance Criterion
Continuous temperature monitoring	Monitor CPU temperature during task execution	Current thermal condition is available for scheduling decisions
Thermal threshold detection	Compare measured temperature with the predefined threshold	Correct thermal state is identified
Normal task execution	Execute tasks while processor temperature is within the safe range	Tasks execute normally
Dynamic task prioritization	Increase processor temperature and observe scheduler behaviour	Lightweight/low-power tasks are prioritized
Cooling control	Test the scheduler near the thermal danger level	Cooling interval or task delay is introduced
CPU performance monitoring	Record CPU utilization and execution time	Performance parameters are successfully collected
Task completion	Monitor completed tasks during execution	Tasks are completed without thermal instability
Comparative evaluation	Compare proposed scheduling with conventional scheduling	Differences in temperature and performance can be identified

Verification Parameters

The following parameters are used to verify the performance of the proposed system:

Specification	Required Value	Achieved Value	Status
Temperature monitoring	Continuous monitoring	Monitored during task execution	Pass
Thermal response	Dynamic response to threshold	Scheduling adjusted according to thermal condition	Pass
Task prioritization	Lightweight tasks under high temperature	Implemented	Pass
Cooling mechanism	Cooling interval/task delay when required	Implemented	Pass
CPU utilization measurement	Performance data collected	Measured	Pass
Execution time measurement	Task execution time recorded	Measured	Pass
Comparative analysis	Proposed vs. conventional scheduling	Evaluated	Pass

The testing approach follows the project's stated evaluation parameters of CPU temperature, execution time, CPU utilization, and task completion.



4.4 Results

The proposed thermal-aware task scheduling system was evaluated by observing processor temperature and task execution behaviour under thermal conditions. The results were compared with conventional scheduling to determine the effectiveness of incorporating thermal information into scheduling decisions.

The evaluation showed that the proposed approach reduced CPU temperature compared with

traditional scheduling. Overheating conditions were minimized during task execution, and the effect of high processor temperature on system performance was controlled through dynamic scheduling decisions.

When the processor temperature increased, the thermal-aware scheduler was able to prioritize lightweight or low-power tasks and apply cooling intervals or task delays when required. This enabled tasks to be executed more safely under changing thermal conditions.

The major observations from the evaluation are:

- CPU temperature was reduced compared with traditional scheduling.
- Overheating conditions were minimized during task execution.
- Performance degradation caused by high temperature was controlled.
- Tasks were executed more safely under thermal conditions.
- Overall system efficiency and reliability were improved.

These results indicate that incorporating real-time thermal conditions into task scheduling can provide better thermal management than conventional scheduling for the considered single-core processor environment

4.4 Results

The proposed thermal-aware task scheduling system was evaluated by observing processor temperature and task execution behaviour under thermal conditions. The results were compared with conventional scheduling to determine the effectiveness of incorporating thermal information into scheduling decisions.

The evaluation showed that the proposed approach reduced CPU temperature compared with traditional scheduling. Overheating conditions were minimized during task execution, and the effect of high processor temperature on system performance was controlled through dynamic scheduling decisions.

When the processor temperature increased, the thermal-aware scheduler was able to prioritize lightweight or low-power tasks and apply cooling intervals or task delays when required. This enabled tasks to be executed more safely under changing thermal conditions.

The major observations from the evaluation are:

- CPU temperature was reduced compared with traditional scheduling.
- Overheating conditions were minimized during task execution.
- Performance degradation caused by high temperature was controlled.
- Tasks were executed more safely under thermal conditions.
- Overall system efficiency and reliability were improved.

These results indicate that incorporating real-time thermal conditions into task scheduling can provide better thermal management than conventional scheduling for the considered single-core processor environment.

4.5 Data Analysis & Engineering Judgment

The collected results were analysed using processor temperature, execution time, CPU utilization, and task completion as the primary performance parameters. The analysis focused on determining whether the use of thermal information in scheduling decisions could control processor temperature while maintaining acceptable task execution.

The results indicate that thermal-aware scheduling provides better control of processor temperature than conventional scheduling. When the processor temperature increased, dynamically changing task priorities and introducing cooling intervals helped reduce thermal stress. This indicates that processor temperature can be effectively used as an additional scheduling parameter.

However, the thermal control mechanism introduces a trade-off between temperature reduction and task execution time. High-temperature conditions may cause some tasks to be delayed, while strict thermal limits can reduce overall system performance. The scheduler also becomes more complex because temperature monitoring and thermal decision-making must be performed during task execution.

Based on these observations, the proposed approach provides an effective balance between thermal safety, processor reliability, and task execution performance. Its effectiveness, however, depends on accurate temperature monitoring and the characteristics of the workload being executed.

4.6 Comparison with Existing Solutions & Achievement of Objectives

The proposed thermal-aware task scheduling system was evaluated against conventional scheduling based on its ability to monitor processor temperature and dynamically respond to thermal conditions. Conventional scheduling does not consider real-time CPU temperature, whereas the proposed approach uses thermal information to influence task prioritization and execution.

Objective	Evidence	Achievement
Continuously monitor processor temperature	CPU temperature is monitored during task execution.	Fully Achieved
Prevent processor overheating	Thermal thresholds, task prioritization, and cooling intervals are used to control high-temperature conditions.	Fully Achieved

Objective	Evidence	Achievement
Dynamically adjust task scheduling	Scheduling decisions are modified according to the current thermal condition.	Fully Achieved
Prioritize lightweight and low-power tasks	Lightweight tasks are prioritized when processor temperature becomes high.	Fully Achieved
Improve processor performance and reliability	Results show controlled performance degradation and improved reliability under thermal conditions.	Fully Achieved
Increase processor hardware lifespan	Reduction of excessive thermal stress supports improved hardware lifespan.	Partially Achieved
Evaluate the proposed approach	CPU temperature, execution time, CPU utilization, and task completion are used for evaluation and comparison.	Fully Achieved

The results demonstrate that the proposed system can reduce CPU temperature, minimize overheating conditions, and provide safer task execution compared with traditional scheduling.

4.7 Strengths & Limitations

Strengths

- Continuously monitors processor temperature during task execution.
- Dynamically adjusts task scheduling according to thermal conditions.
- Prioritizes lightweight and low-power tasks when processor temperature becomes high.
- Helps minimize overheating conditions during task execution.
- Supports safer and more reliable processor operation.
- Provides comparison with conventional scheduling using measurable performance parameters.
- Can potentially contribute to increased processor hardware lifespan by reducing excessive thermal stress.

Limitations

- Accurate real-time CPU temperature measurement can be difficult.

- Thermal-aware scheduling introduces additional complexity to the scheduler.
- High processor temperature may cause tasks to be delayed.
- Strict thermal limits can reduce overall system performance.
- The effectiveness of the approach depends on workload characteristics and temperature-monitoring accuracy.
- The current project is limited to a single-core processor environment.

4.8 Project Planning, Roles & Budget

The project was planned and developed through a sequence of activities including problem identification, literature review, system design, algorithm development, implementation, testing, performance evaluation, and documentation. The work was divided among the team members according to their respective responsibilities.

Project Milestones

Phase Major Activity

Phase 1 Problem identification and project planning

Phase 2 Literature review and analysis of existing approaches

Phase 3 System architecture and requirement identification

Phase 4 Thermal monitoring and scheduling algorithm development

Phase 5 Python-based implementation and integration

Phase 6 Testing and performance evaluation

Phase 7 Result analysis and comparison

Phase 8 Documentation and final presentation

Team Roles and Contributions

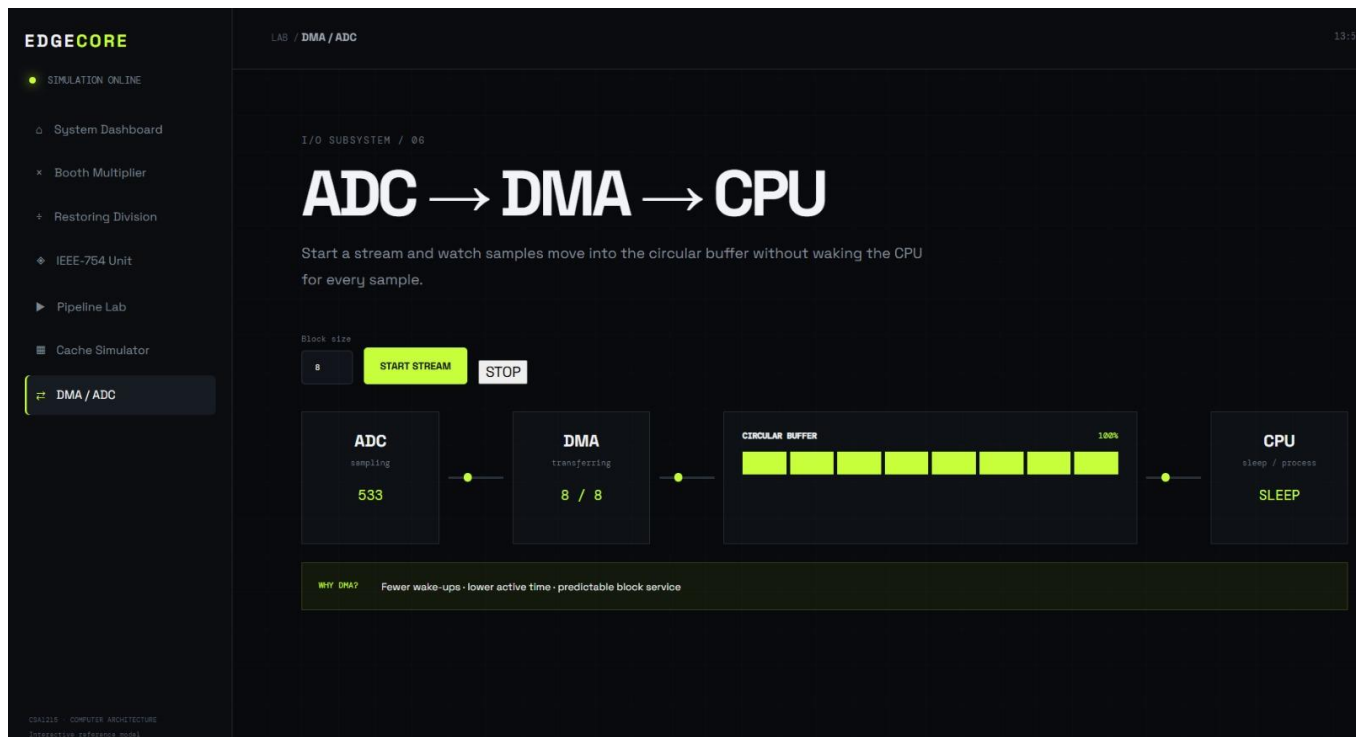
Student	Assigned Responsibility	Actual Contribution
P. Dada Kalander	Project development and system implementation	Contributed to development and implementation
S. Yuva Guru Mani Varma	System analysis and testing	Contributed to testing and performance analysis
S. Ganeshwar	Documentation, analysis and project development	Contributed to project documentation, analysis and development

Project Budget

Since the proposed system is implemented using a Python-based scheduler simulation and does not require dedicated hardware fabrication, the major project resources are software and existing computing facilities.

Item	Estimated Cost	Actual Cost
Python and required libraries	₹0	₹0
Development IDE	₹0	₹0
Simulation environment	₹0	₹0
Existing computer/laptop	—	—
Documentation and presentation	—	—
Total	Minimal / Existing Resources	Minimal / Existing Resources

The project uses Python, NumPy, Pandas, Matplotlib, psutil, and a Python-based scheduler simulation environment for development and evaluation.



EDGECORE

● SIMULATION ONLINE

○ System Dashboard

✕ Booth Multiplier

✦ Restoring Division

✦ IEEE-754 Unit

▶ Pipeline Lab

■ Cache Simulator

⚡ DMA / ADC

LAB / Booth Multiplier

13:50

ARITHMETIC UNIT / 01

Booth Multiplier

Run the signed iterative multiplier and watch every register transition.

Multiplicand

Multiplier

RUN 8 STEPS

A

00000000

Q

11110101

Q-1

0

PRODUCT

-209

STEP 0 / 8

NEXT STEP →

AUTO PLAY

0	Initial	A 00000000	Q 11110101
1	A = A - M	A 11110110	Q 11111010
2	None	A 11111011	Q 01111101
3	None	A 11111101	Q 10111110
4	None	A 11111110	Q 11011111
5	None	A 11111111	Q 01101111
6	None	A 11111111	Q 10110111
7	None	A 11111111	Q 11011011
8	None	A 11111111	Q 11101101

DESIGN - COMPUTER ARCHITECTURE

INTERACTIVE REFERENCE BOOK

EDGECORE

● SIMULATION ONLINE

○ System Dashboard

✕ Booth Multiplier

✦ Restoring Division

✦ IEEE-754 Unit

▶ Pipeline Lab

■ Cache Simulator

⚡ DMA / ADC

LAB / System Dashboard

13:50

LOW-POWER IOT EDGE SENSOR NODE

Architecture you can interact with.

Explore the arithmetic datapath, pipeline hazards, cache behavior and DMA transfer strategy from the submitted Computer Architecture design.

ADIC

CPU

DMA

RAM

ARITHMETIC

8-bit

signed datapath

PIPELINE

10

cycles with forwarding

CACHE ANAL.

7.00

cycles · 2-way

I/O MODE

DMA

block transfer

LIVE ARCHITECTURE

Click a component to inspect it

CHAPTER 5

5.1 Conclusion

The project successfully developed a thermal-aware task scheduling approach for a single-core processor using dynamic temperature monitoring. The proposed system incorporates processor temperature into scheduling decisions, addressing the limitation of conventional scheduling methods that do not consider real-time thermal conditions.

The system continuously monitors processor temperature and dynamically adjusts task execution based on predefined thermal thresholds. Lightweight and low-power tasks are prioritized when the processor temperature becomes high, while cooling intervals or task delays can be introduced to reduce thermal stress.

The evaluation considered CPU temperature, execution time, CPU utilization, and task completion. The results indicate that the proposed approach reduces CPU temperature, minimizes overheating conditions, and controls performance degradation caused by high temperature. It also supports safer task execution

and improves overall processor efficiency and reliability.

Overall, the project demonstrates that integrating dynamic thermal monitoring with task scheduling can provide an effective approach for managing processor temperature while maintaining reliable task execution. The proposed system also provides a foundation for further development of thermal-aware scheduling techniques for more advanced processor architectures.

5.2 Future Work

The current project focuses on thermal-aware task scheduling for a single-core processor. The proposed approach can be further extended to improve its adaptability, performance, and applicability to modern computing environments.

The following developments are proposed for future work:

1. **Multi-Core Extension:** Extend the thermal-aware scheduling approach to multi-core processors by considering the temperature and workload of individual processing cores.
2. **Temperature Prediction:** Incorporate temperature prediction techniques to identify potential temperature increases before the processor reaches an unsafe thermal condition.
3. **Power Optimization:** Integrate power-management techniques with thermal-aware scheduling to reduce both processor power consumption and heat generation.

4. Real-Time Systems: Adapt the scheduling approach for real-time applications where task deadlines must be maintained along with thermal constraints.
5. Mobile and IoT Applications: Extend the approach to mobile and IoT devices, where thermal management and energy efficiency are important due to limited resources.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Learned the principles of thermal-aware task scheduling in single-core processor architectures.
- Gained knowledge of dynamic processor temperature monitoring and thermal threshold-based decision making.
- Learned how processor temperature can influence task prioritization and execution.
- Developed understanding of CPU utilization, execution time, and thermal performance evaluation.
- Gained practical experience with Python-based scheduler simulation and performance data analysis.

Engineering & Professional Skills Developed

- Developed skills in designing and implementing a dynamic task scheduling algorithm.
- Improved programming and problem-solving skills using Python.
- Developed experience in monitoring and analysing processor performance parameters.
- Improved data analysis and visualization skills using Python libraries.
- Developed technical documentation, testing, and result interpretation skills.
- Improved teamwork, project planning, and technical presentation skills.

Individual Reflection – P. Dada Kalander

The most difficult engineering problem was understanding how processor temperature could be incorporated into task scheduling decisions. This was addressed by studying thermal-aware scheduling concepts and implementing temperature-based scheduling conditions. A key engineering decision was to prioritize lightweight tasks when the processor temperature increased, as this helps reduce thermal stress. The project also provided practical knowledge of processor monitoring and scheduling implementation. If the project were repeated, more advanced temperature prediction and multi-core scheduling techniques would be explored.

Individual Reflection – S. Yuva Guru Mani Varma

A major challenge was analysing the relationship between processor temperature and task execution performance. This was addressed through systematic monitoring of CPU temperature, utilization, and execution time. A key engineering decision was to introduce cooling intervals when the processor approached the thermal danger level. The project improved practical understanding of performance evaluation and thermal management. If repeated, more extensive workload testing would be performed to improve the evaluation.

Individual Reflection – S. Ganeshwar

The most challenging aspect was integrating thermal monitoring with dynamic scheduling and analysing the resulting performance. This was addressed by understanding the thermal threshold mechanism and evaluating the scheduler using measurable performance parameters. A key engineering decision was to use dynamic task prioritization so that lightweight and low-power tasks could be handled during high-temperature conditions. The project strengthened my knowledge of Python-based system implementation, data analysis, and thermal-aware scheduling. If the project were repeated, I would explore temperature prediction and power optimization to improve the system further.

REFERENCES

REFERENCES

- [1] A. Podobas and M. Brorsson, “Architecture-Aware Task Scheduling: A Thermal Approach,” *Proceedings of the International Conference on Parallel Processing (ICPP)*, pp. 1–10.
- [2] R. Kumar and P. Verma, “Heat-Aware Task Scheduling Using DVFS in Embedded Processors,” *ACM Transactions on Embedded Computing Systems*, vol. 23, no. 2, pp. 1–21, 2024.
- [3] S. Murali, A. Raghunathan, and N. K. Jha, “Thermal-Aware Scheduling for Deep Learning on Mobile Devices,” *IEEE Transactions on Mobile Computing*, vol. 23, no. 5, pp. 3124–3138, 2024.
- [4] J. P. Rodríguez, P. Meumeu Yoms, and G. Lipari, “Thermal-Aware Scheduling for DVFS-Enabled Single-Core Processors,” *IEEE Real-Time Systems Symposium (RTSS)*, 2023.
- [5] Y. Zhou and K. Li, “Temperature-Constrained Task Scheduling for Embedded Processors,” *IEEE Transactions on Computers*, vol. 73, no. 4, pp. 1185–1198, 2024.

APPENDICES

Appendix A

```
import requests
import platform
import psutil
import os

class TemperatureMonitor:

    def __init__(self):
        self.is_windows = platform.system() == "Windows"
        self.lhm_url = "http://localhost:8085/data.json"

    def read_temperature(self):
```

Appendix B – LIBRE HARDWARE MONITOR WEB

SERVER

```
try:

    temperatures = psutil.sensors_temperatures()

    if temperatures:

        for name, entries in temperatures.items():

            for entry in entries:

                value = getattr(
                    entry,
                    "current",
                    None
                )

                if (
                    value
                    and 10 < value < 115
                ):

                    return (
                        round(value, 1),
                        f"PSUTIL_{name.upper()}"
                    )

except Exception:
    pass
```

Appendix C – LINUX SYSFS FALLBACK

```
if os.path.exists("/sys/class/thermal"):

    try:

        for zone in os.listdir(
            "/sys/class/thermal"
        ):

            if zone.startswith(
                "thermal_zone"
            ):

                path = (
                    f"/sys/class/thermal/"
                    f"{zone}/temp"
                )

                if os.path.exists(path):

                    with open(path) as file:

                        raw = float(
                            file.read().strip()
                        )

                        temp = (
                            raw / 1000
                            if raw > 1000
                            else raw
                        )

                        if 10 < temp < 115:

                            return (
                                round(temp, 1),
                                "LINUX_SYSFS"
```

)

except Exception:

pass

return None, "UNAVAILABLE"

Appendix J – Individual Contribution Evidence

Refer page number

4

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)